

GHOST: A C++ framework to model and solve combinatorial optimization problems

Florian Richoux ¹✉

¹ Artificial Intelligence Research Center, National Institute of Advanced Industrial Science and Technology (AIST), Japan  ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Fangzhou Xie](#) 

Reviewers:

- [@alexzerg](#)
- [@cprudhom](#)

Submitted: 14 September 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

GHOST (General meta-Heuristic Optimization Solving Toolkit) is an open-source C++17 framework for modeling and solving discrete combinatorial problems. It supports Constraint Satisfaction Problems (CSPs), Constrained Optimization Problems (COPs), and related formulations in which error functions quantify constraint violations. Users declare variables, constraints, and, optionally, an objective function, while GHOST supplies a constraint-based local search solver.

GHOST was initiated in 2014 for a *StarCraft: Brood War* bot, where all decisions for a game frame had to be computed in less than 100 ms. It has since supported research on decision making with short time budgets, terrain analysis, automatic error-function learning, Quadratic Unconstrained Binary Optimization (QUBO) reformulation, and combinations of combinatorial optimization with Monte Carlo Tree Search ([Antuori & Richoux, 2019](#); [Richoux, 2022, 2025](#); [Richoux et al., 2023, 2016](#); [Richoux & Baffier, 2023](#)). Its benchmark repository includes classic combinatorial (optimization) problems, such as Sudoku, N-Queens, the knapsack problem, magic squares, the quadratic assignment problem, the traveling salesman problem, the traveling tournament problem, and vertex cover ([Richoux, 2026a](#)).

Statement of need

Combinatorial optimization software involves trade-offs between modeling convenience, control over search, solution quality, and response time. Complete solvers, i.e., solvers exploring the whole search space, are valuable when optimality or infeasibility proofs are required, but such guarantees may be incompatible with interactive applications. Conversely, implementing a specialized metaheuristic provides control but requires substantial effort and often entangles the problem model with the search procedure.

GHOST addresses the intermediate need for a reusable native C++ framework that accepts constraint-based models and provides a ready-to-use metaheuristic solver. Its intended users are researchers and C++ developers who need feasible and improving solutions under limited computation budgets, wish to test alternative models without repeatedly implementing a solver, or need to embed combinatorial decision making in a C++ system. Relevant domains include decision-making, games, scheduling, logistics, routing, assignment, and resource allocation.

Constraints in GHOST have non-negative error functions, for which zero denotes satisfaction. The solver therefore obtains information about how far a candidate is from feasibility rather than only a Boolean signal. Because Adaptive Search is a metaheuristics, GHOST is an anytime solver: it returns its best solution when the allocated time expires, but cannot prove optimality or infeasibility.

State of the field

Constraint programming systems such as Choco (Prud'homme & Fages, 2022) and OR-Tools CP-SAT (Google, n.d.) provide expressive models and systematic solving techniques. They are natural choices when strong propagation, bounds, or proof-oriented solving are principal requirements. Comet introduced language-level support for constraint-based local search (Van Hentenryck & Michel, 2005), while OscaR.cbls provides a Scala framework for composing models, invariants, neighborhoods, and search procedures (CETIC, n.d.).

The Constraint programming community mainly proposes different languages to declare problem models, such as XCSP3 and MiniZinc, and compiled a large catalogue of more than 400 global constraints that users are encouraged to use in order to achieve the best solver performance. Originally, these global constraints aimed to isolate the most frequently used constraints and optimize their internal mechanisms, so that users would not have to reinvent the wheel and would have a comprehensive set of various constraints at their disposal to express their problems. However, we believe that the adoption of specialized languages—rather than libraries written in general-purpose programming languages—combined with an overly broad set of global constraints requiring genuine expertise to master, has hindered rather than helped the democratization of constraint programming in both industry and scientific research, paving the way for other approaches.

This is why GHOST occupies a different design point from other tools in the field. It is a lightweight C++ framework intended for direct integration into native applications and for problems with short, explicit time budgets. Users provide a constraint-level model while the framework supplies generic search control. This reduces the need to implement neighborhood exploration and metaheuristic logic for every application, while retaining extensibility through custom constraints and objective functions. Its priorities are ease of modeling and use, then execution speed. Users can define their own, custom constraints, but also use predefined constraints among a restricted set of the most common ones, such as AllDifferent, Linear Equation, and their variants. GHOST therefore complements rather than replaces exact solvers and more configurable local-search toolkits.

Software design

The main GHOST solver implements Adaptive Search, a constraint-based local search algorithm (Codognet & Diaz, 2001). From a current assignment, its neighborhood contains every candidate reachable by changing one variable's value. It evaluates the constraint error functions and project on variables the error of each constraint a variable interact with. The search for solutions is guided by changing the assignment of variables with the largest error. After finding a feasible solution, it seeks better solutions according to the objective. When search appears trapped in a local optimum, it performs either a reset, randomly reassigning some variables, or a restart, randomly reassigning all variables.

The architecture separates problem models from search, rooting its philosophy into the “holy grail” of Constraint Programming: “Users declare the problem, the computer solves it.” A model defines variables and domains, constraints, and an optional objective. Satisfaction and optimization share these abstractions, and permutation and non-permutation representations are supported.

In addition of the constraint-based local search solver, a simple non-backtracking complete solver was later added to enumerate all solutions of an instance. This was motivated by the AI agent playing the board game *boop*. on an Android app, where enumeration was needed within Monte Carlo Tree Search (Richoux, 2025). The framework's evolution is documented in its change log (Richoux, 2026b).

GHOST is built with CMake and distributed under GNU GPL v3. The repository includes

88 installation instructions for GNU/Linux, macOS, and Windows, tutorials, an introduction to
89 constraint programming, Doxygen API documentation, automated tests, continuous integration,
90 releases, a PDF cheat sheet, and a 30-minute introductory video (Richoux, n.d., 2026b). A
91 separate repository provides executable benchmarks (Richoux, 2026a).

92 Research impact statement

93 GHOST has been developed openly since 2014. The original work modeled target selection,
94 wall-in construction, build-order planning, and resource allocation for real-time strategy games
95 (Richoux et al., 2016). Further work studied robustness and flexibility on pathfinding and
96 alternative resource-allocation models (Fradin & Richoux, 2015).

97 The software has subsequently enabled research on decision making under uncertainty and the
98 microPhantom bot for *microRTS* (Antuori & Richoux, 2019; Richoux, 2020), constraint-based
99 terrain analysis in *StarCraft* 1 and 2 (Richoux, 2022), learning interpretable error functions
100 (Richoux & Baffier, 2021, 2023), learning QUBO constraint representations (Richoux et
101 al., 2023), and injecting combinatorial optimization into Monte Carlo Tree Search for *boop*.
102 (Richoux, 2025).

103 GHOST is integrated into pobo, an Android implementation of *boop*; Taunt, a C++ terrain-
104 analysis library for *StarCraft* 1 and 2; and microPhantom, a bot for *microRTS*. It has also
105 supported research systems for learning constraint error functions and QUBO reformulations.
106 These applications influenced its evolution, including complete solution enumeration.

107 AI usage disclosure

108 Microsoft 365 Copilot was used to prepare and revise this manuscript from the repositories,
109 JOSS guidance, bibliographic sources, and information supplied by the author. The author
110 reviewed the text and remains responsible for it. No generative AI was used to produce the
111 GHOST source code or documentation.

112 Acknowledgements

113 GHOST was developed while the author held positions at the French National Centre for Scientific
114 Research (CNRS), Nantes University, and the National Institute of Advanced Industrial Science
115 and Technology (AIST). These organizations did not provide dedicated funding specifically for
116 GHOST.

117 References

- 118 Antuori, V., & Richoux, F. (2019). Constrained optimization under uncertainty for decision-
119 making problems: Application to real-time strategy games. *IEEE Congress on Evolutionary*
120 *Computation*. <https://richoux.fr/publications/antuori2019cec.pdf>
- 121 CETIC. (n.d.). *Oscar.cbls*. <https://github.com/cetic/oscar-cbls>
- 122 Codognet, P., & Diaz, D. (2001). Yet another local search method for constraint solving.
123 *Stochastic Algorithms: Foundations and Applications*, 73–90.
- 124 Fradin, J., & Richoux, F. (2015). Robustness and flexibility of GHOST. *AIIDE Workshop on*
125 *Artificial Intelligence for Strategy Games*. [https://richoux.fr/publications/fradin2015aiidews.](https://richoux.fr/publications/fradin2015aiidews.pdf)
126 [pdf](https://richoux.fr/publications/fradin2015aiidews.pdf)
- 127 Google. (n.d.). *OR-Tools*. <https://developers.google.com/optimization/>

- 128 Prud'homme, C., & Fages, J.-G. (2022). Choco-solver: A java library for constraint
129 programming. *Journal of Open Source Software*, 7(78), 4708. [https://doi.org/10.21105/](https://doi.org/10.21105/joss.04708)
130 [joss.04708](https://doi.org/10.21105/joss.04708)
- 131 Richoux, F. (n.d.). *Introduction to GHOST*. YouTube video. <https://youtu.be/GylF0125KZQ>
- 132 Richoux, F. (2025). Injecting combinatorial optimization into MCTS: Application to the
133 board game boop. *Applications of Evolutionary Computation*, 15613, 525–540. https://doi.org/10.1007/978-3-031-90065-5_32
134 https://doi.org/10.1007/978-3-031-90065-5_32
- 135 Richoux, F. (2022). Terrain analysis in StarCraft 1 and 2 as combinatorial optimization. *IEEE*
136 *Congress on Evolutionary Computation*, 1–8. [https://doi.org/10.1109/CEC55065.2022.](https://doi.org/10.1109/CEC55065.2022.9870230)
137 [9870230](https://doi.org/10.1109/CEC55065.2022.9870230)
- 138 Richoux, F. (2020). microPhantom: Playing microRTS under uncertainty and chaos. *IEEE*
139 *Conference on Games*. <https://richoux.fr/publications/richoux2020microphantom.pdf>
- 140 Richoux, F. (2026a). *GHOST Benchmarks*. <https://github.com/richoux/GHOST-benchmarks>
- 141 Richoux, F. (2026b). *GHOST: General meta-heuristic optimization solving toolkit*. <https://github.com/richoux/GHOST>
142 <https://github.com/richoux/GHOST>
- 143 Richoux, F., & Baffier, J.-F. (2021). Error function learning with interpretable compositional
144 networks for constraint-based local search. *Genetic and Evolutionary Computation*
145 *Conference Companion*, 137–138. <https://doi.org/10.1145/3449726.3459464>
- 146 Richoux, F., & Baffier, J.-F. (2023). Automatic error function learning with interpretable
147 compositional networks. *Annals of Mathematics and Artificial Intelligence*. <https://doi.org/10.1007/s10472-022-09829-8>
148 <https://doi.org/10.1007/s10472-022-09829-8>
- 149 Richoux, F., Baffier, J.-F., & Codognet, P. (2023). Learning QUBO models for quantum
150 annealing: A constraint-based approach. *Computational Science – ICCS 2023*, 14077,
151 153–167. https://doi.org/10.1007/978-3-031-36030-5_12
- 152 Richoux, F., Uriarte, A., & Baffier, J.-F. (2016). GHOST: A combinatorial optimization
153 framework for real-time problems. *IEEE Transactions on Computational Intelligence and*
154 *AI in Games*, 8(4), 377–388. <https://doi.org/10.1109/TCIAIG.2016.2573199>
- 155 Van Hentenryck, P., & Michel, L. (2005). *Constraint-based local search*. MIT Press.